

input and predicts a target word that is closely connected to the input words' context. The skip-gram design, on the other hand, takes one word as input and anticipates its closely related context terms.

CBOW is faster and finds better numerical representations for common words, although skip-gram can represent rare words effectively. Word2vec models excel at capturing semantic connections between words. Consider the relationship between a country and its capital, such as France's capital, Paris, and Germany's capital, Berlin. These models are best at doing semantic analysis, which is useful for making recommendations.

Applicability of word2vec models in building product recommendations:

Word2vec uses the sequential nature of the text as a fundamental property of a natural language to produce vector representations of text. There is a word sequence in every sentence or phrase. We would have great difficulty understanding the text without this sequence.

One such example is the purchases made by consumers on e-commerce websites online. There is generally a pattern in the buying behaviour of consumers. For instance, a person involved in sports-related activities might have an online buying pattern similar to what is shown in Figure 8.

We can quickly locate related products if we can represent each of these things as a vector. So, if a user is looking at a product online, we can quickly offer related things to him

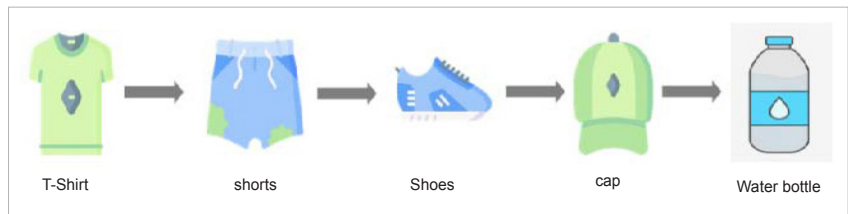


Figure 8: Purchase history

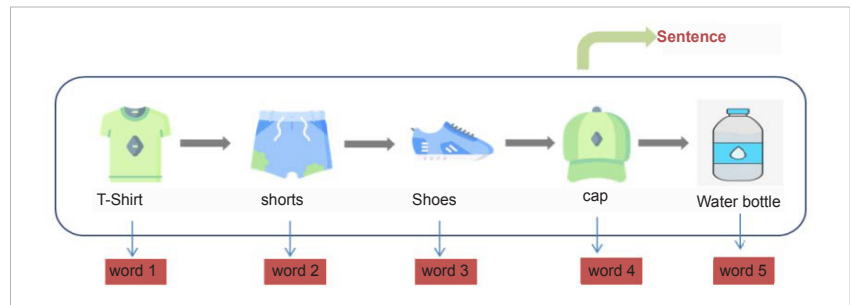


Figure 9: Customer's purchasing history as a sentence

or her based on the product's vector similarity score. However, how can we obtain a vector representation of these items? Is it possible to obtain these vectors using the word2vec model? Yes, it is! Consider a customer's purchasing history as a sentence, with the products as the words, as shown in Figure 9.

So we have seen how a recommender system works and the mathematics behind it. We have also covered some basics of the ML being used in such a system. In the next part of this two-part series of articles, we will see how we can apply the knowledge learnt here to build a recommender system in Python. **END** 🐧

References

- [1] https://en.wikipedia.org/wiki/Recommender_system
- [2] <https://www.tensorflow.org/tutorials/text/word2vec>
- [3] <https://www.opensourceforu.com/2020/08/natural-language-processing-simplified>
- [4] <https://www.opensourceforu.com/2021/11/the-significance-of-neural-networks-in-nlp>

By: Dibyendu Banerjee

The author is a senior architect at Cognizant's AI and analytics practice, with more than 15 years of IT experience. His areas of expertise are Python, R, Java and other open source technologies.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU